

Aufgabe 1. Wenden Sie auf das lineare Gleichungssystem $Ax = b$,

$$A = \begin{pmatrix} 4 & 2 & 0 \\ 2 & 4 & 2 \\ 0 & 2 & 4 \end{pmatrix}, \quad b = \begin{pmatrix} 2 \\ 0 \\ 2 \end{pmatrix}$$

das Einzelschrittverfahren an.

- (a) Berechnen Sie für $x_0 = (0, 0, 0)^T$ die Iterierten $x_1, \dots, x_4$. Geben Sie jeweils 6 führende Dezimalstellen an.
- (b) Bestimmen Sie die Iterationsmatrix B_{GS} sowie $\|B_{GS}\|_\infty$. Konvergiert das Verfahren? Fällt der Fehler $\|e_k\|_\infty$ monoton?

Aufgabe 2. Betrachten Sie das lineare Gleichungssystem $Ax = b$ mit

$$A = \begin{pmatrix} 2 & 3 \\ 1 & 2 \end{pmatrix}, \quad b = \begin{pmatrix} 203 \\ 102 \end{pmatrix}.$$

- (a) Berechnen Sie (mit dem Taschenrechner, auf 6 führende Stellen genau) die Iterierten $x_1, \dots, x_5$ für das Jacobi- sowie das Gauß-Seidel-Verfahren zum Startwert $x_0 = (0, 0)^T$.
- (b) Bestimmen Sie die Iterationsmatrizen B_J und B_{GS} der beiden Verfahren sowie deren Spektralradien.
- (c) Würde sich beim Jacobi-Verfahren der Einsatz eines Relaxationsparameters lohnen?

Aufgabe 3. Betrachten Sie die Matrix

$$A = \begin{pmatrix} 0 & -4 & 0 \\ 0 & 2 & -2 \\ 0 & -1 & 1 \end{pmatrix}.$$

- (a) Schätzen Sie mit Hilfe des Satzes von Gerschgorin die Lage der Eigenwerte von A ab.
- (b) Berechnen Sie (von Hand) die Eigenwerte von A .
- (c) Bestimmen Sie mit Hilfe der Vektoriteration zum Startwert $x_0 = (0, 1, 0)^T$ Näherungen für einen Eigenwert *und* einen Eigenvektor der Matrix A . Iterieren Sie so lange, bis sich bei der Näherung des Eigenwerts die führenden 6 Dezimalstellen nicht mehr ändern.

Aufgabe 4. Geben Sie eine Implementierung des Bisektions-Verfahrens in Form einer MATLAB bzw. SCILAB Funktion mit Schnittstelle

`x = bisektion(f,a,b,n)`

an. Dabei ist **f** ein Funktionszeiger auf die zu untersuchende Funktion, **a, b** das Startintervall und **n** die Anzahl an Iterationen.

Aufgabe 5. Betrachten Sie die Funktion

$$f(x) = \sin(x) - \cos(x).$$

- (a) Zeigen Sie, dass f in $[0, 2]$ mindestens eine Nullstelle besitzt.
- (b) Ermitteln Sie mit dem Bisektionsverfahren ein Teilintervall von $[0, 2]$ mit Länge $\frac{1}{4}$, das eine Nullstelle von f enthält.
- (c) Verbessern Sie das Resultat aus (b) mit dem Newton-Verfahren. Verwenden Sie als Startwert die Intervallmitte des *letzten* Intervalls aus (b). Iterieren Sie so lange, bis sich die ersten 6 führenden Ziffern nicht mehr verändern.

Aufgabe 6. Betrachten Sie die folgenden Punkte (x_i, y_i) :

i	0	1	2	3
x_i	-1	0	1	3
y_i	5	3	5	21

- (a) Bestimmen Sie das zugehörige Newton-Interpolationspolynom. Benutzen Sie dividierte Differenzen

$$d_{ii} = f(x_i) \quad i = 0, 1, \dots \quad d_{nm} = \frac{d_{n \ m+1} - d_{n-1 \ m}}{x_n - x_m} \quad n > m.$$

- (b) Approximieren Sie die Punkte durch ein Polynom der Form

$$p(x) = a + b \cdot x^2$$

nach der Methode der kleinsten Fehlerquadrate.

Aufgabe 7.

Berechnen Sie

$$\int_0^4 \frac{x}{1+x} dx$$

mit Hilfe der summierten Trapezregel T_1 , T_2 , T_4 mit 1, 2 bzw. 4 Trapezen.

Aufgabe 8. Wir betrachten die Integrationsaufgabe

$$\int_0^1 \frac{1}{1+x^2} dx. \tag{1}$$

- (a) Zeigen Sie, dass

$$\int_0^1 \frac{1}{1+x^2} dx = \frac{\pi}{4}.$$

- (b) Geben Sie für das obige Integral die Rechteckformeln für die Ober- bzw. Untersumme O_n, U_n für n gleichbreite Teilintervalle an. Wie groß ist der maximale Fehler $D_n = O_n - U_n$?
- (c) Berechnen Sie O_n, U_n, D_n für 10 Teilintervalle.
- (d) Wieviele Teilintervalle sind nötig, um $\frac{\pi}{4}$ auf mindestens 3 Nachkommastellen genau angeben zu können (benutzen Sie (b))?

- (e) Berechnen Sie eine Näherung von (1) mit der *einfachen* Simpson-Regel. Wie groß ist der absolute Fehler?
- (f) Wenden Sie die *summierte* Simpson-Regel mit Streifenzahl 10 auf (1) an. Wie groß ist der absolute Fehler?
- (g) Berechnen Sie mit dem Gauß-Verfahren $G_1(f)$ eine Näherung von (1) und bestimmen Sie den absoluten Fehler.

Aufgabe 9. Zur Gauß-Integration benutzt man eine Approximation

$$\int_a^b f(x) dx \approx \sum_{i=1}^n w_i f(x_i), \quad (2)$$

wobei die Gewichte w_i und Stützstellen x_i so bestimmt werden, dass die rechte Seite von (2) für Polynome möglichst hohen Grades den exakten Wert liefert.

Da Polynome Linearkombinationen von Monomen x^j sind, müssen die Gewichte w_i und Stützstellen x_i Lösungen des nichtlinearen Gleichungssystems

$$\int_a^b x^j dx = \sum_{i=1}^n w_i x_i^j, \quad j = 0, \dots, 2n-1 \quad (3)$$

sein.

- (a) Stellen Sie für $n = 2$, $-a = b = 1$ das System (3) auf und zeigen Sie, dass $w_1 = w_2 = 1$ und $-x_1 = x_2 = \frac{1}{\sqrt{3}}$ eine Lösung ist.
- (b) Wenden Sie mit den Ergebnissen aus (a) die Formel (2) auf $\int_{-1}^1 e^x dx$ an und vergleichen Sie das Ergebnis mit dem exakten Wert. Wie groß ist der absolute Fehler?
- (c) Vergleichen Sie den Fehler aus (b) mit den Fehlern die sich bei Anwendung der *einfachen* Trapez- bzw. Simpson-Regel ergeben.
- (d) Wieviele Teilintervalle m müssen bei der summierten Trapezregel berücksichtigt werden, damit der absolute Fehler e_a kleiner ist als in (b)? Es gilt

$$e_a \leq \frac{M(b-a)^3}{12m^2}, \quad M = \max_{x \in [a,b]} |f''(x)|.$$

- (e) Berechnen Sie das Integral nach (d). Geben Sie die Formel sowie alle benutzten Funktionswerte an. Wie groß ist der absolute Fehler?
- (f) Berechnen Sie unter ausschließlicher Verwendung der Funktionswerte aus (e) das Integral mit der *summierten* Simpson-Regel und den zugehörigen absoluten Fehler.

Geben Sie alle Ergebnisse und Zwischenergebnisse mit voller Taschenrechnergenauigkeit an.

Aufgabe 10. Betrachten Sie das Anfangswertproblem

$$\dot{x} = x^2, \quad x(0) = 0.1$$

- (a) Berechnen Sie eine Näherung von $x(4)$ mit Hilfe des Euler-Verfahrens mit Schrittweite $h = 1$. Geben Sie jeweils die Werte an allen Zwischenpunkten an.

(b) Berechnen Sie mit Hilfe des klassischen Runge-Kutta-Verfahrens

$$k_1 = f(t_k, x_k),$$

$$k_2 = f\left(t_k + \frac{h}{2}, x_k + \frac{h}{2}k_1\right),$$

$$k_3 = f\left(t_k + \frac{h}{2}, x_k + \frac{h}{2}k_2\right),$$

$$k_4 = f(t_k + h, x_k + hk_3),$$

$$x_{k+1} = x_k + h\left(\frac{1}{6}k_1 + \frac{2}{6}k_2 + \frac{2}{6}k_3 + \frac{1}{6}k_4\right),$$

mit Schrittweite $h = 4$ eine Näherung von $x(4)$. Geben Sie die Werte der k_i an.

Geben Sie bei allen Werten mindestens 6 führenden Dezimalstellen an.

```
function x= bisektion(f,a,b,n)

for k= 1:n
    x= (a+ b)/ 2;

    fx= evstr(f + '(x)');
    fa= evstr(f + '(a)');

    if (fx*fa) < 0
        b= x;
    else
        a= x;
    end
end

endfunction
```

Aufgabe 1

```
> restart:
with(linalg):
ll:=2:dd:=4:rr:=2:
DD:=dd*matrix([[1,0,0],[0,1,0],[0,0,1]]):
E:=-ll*matrix([[0,0,0],[1,0,0],[0,1,0]]):
F:=-rr*matrix([[0,1,0],[0,0,1],[0,0,0]]):
A:=evalm(DD-E-F);
b:=vector([dd-rr,ll-dd+rr,-ll+dd]);
xx:=linsolve(A,b):
```

$$A := \begin{bmatrix} 4 & 2 & 0 \\ 2 & 4 & 2 \\ 0 & 2 & 4 \end{bmatrix}$$

$$b := \begin{bmatrix} 2 & 0 & 2 \end{bmatrix}$$

(1)

Teil (a)

```
> Bgs:=multiply(inverse(evalm(DD-E)),F):
dgs:=multiply(inverse(evalm(DD-E)),b):
x:=vector([0,0,0]);
for i from 1 to 4 do
  x:=evalf(evalm(multiply(Bgs,x)+dgs));
od;
```

$$x := \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$$

$$x := \begin{bmatrix} 0.5000000000 & -0.2500000000 & 0.6250000000 \end{bmatrix}$$

$$x := \begin{bmatrix} 0.6250000000 & -0.6250000000 & 0.8125000000 \end{bmatrix}$$

$$x := \begin{bmatrix} 0.8125000000 & -0.8125000000 & 0.9062500000 \end{bmatrix}$$

$$x := \begin{bmatrix} 0.9062500000 & -0.9062500000 & 0.9531250000 \end{bmatrix}$$

(2)

Teil(b)

```
> h:=inverse(evalm(DD-E)):
```

```
> bgs:=multiply(h,F);
```

$$bgs := \begin{bmatrix} 0 & -\frac{1}{2} & 0 \\ 0 & \frac{1}{4} & -\frac{1}{2} \\ 0 & -\frac{1}{8} & \frac{1}{4} \end{bmatrix}$$

(3)

```
> normbgs:=norm(Bgs,infinity);
```

$$\text{normbgs} := \frac{3}{4}$$

(4)

Da $\rho(Bgs) \leq \text{norm}(Bgs) < 1$ konvergiert das Verfahren

Wegen $\text{norm}(Bgs) < 1$ ist die Konvergenz in der Unendlich-Norm monoton

Aufgabe 2

```
> restart:
with(linalg):
> d:=matrix([[2,0],[0,2]]):
e:=matrix([[0,0],[-1,0]]):
f:=matrix([[0,-3],[0,0]]):
a:=evalm(d-e-f);
b:=multiply(a,vector([100,1]));
```

$$a := \begin{bmatrix} 2 & 3 \\ 1 & 2 \end{bmatrix}$$

$$b := \begin{bmatrix} 203 & 102 \end{bmatrix}$$

(1)

```
> bj:=multiply(inverse(d),e+f):
dj:=multiply(inverse(d),b):
bgs:=multiply(inverse(evalm(d-e)),f):
dgs:=multiply(inverse(evalm(d-e)),b):
```

Teil (a):

```
> x:=vector([0,0]);
for i from 1 to 5 do
  x:=evalf(evalm(multiply(bj,x)+dj))
od;
```

$$x := \begin{bmatrix} 0 & 0 \end{bmatrix}$$

$$x := \begin{bmatrix} 101.5000000 & 51. \end{bmatrix}$$

$$x := \begin{bmatrix} 25.00000000 & 0.25000000 \end{bmatrix}$$

$$x := \begin{bmatrix} 101.1250000 & 38.50000000 \end{bmatrix}$$

$$x := \begin{bmatrix} 43.75000000 & 0.43750000 \end{bmatrix}$$

$$x := \begin{bmatrix} 100.8437500 & 29.12500000 \end{bmatrix}$$

(2)

```
> x:=vector([0,0]);
for i from 1 to 5 do
  x:=evalf(evalm(multiply(bgs,x)+dgs))
od;
```

$$x := \begin{bmatrix} 0 & 0 \end{bmatrix}$$

$$x := \begin{bmatrix} 101.5000000 & 0.2500000000 \end{bmatrix}$$

$$x := \begin{bmatrix} 101.1250000 & 0.4375000000 \end{bmatrix}$$

$$x := \begin{bmatrix} 100.8437500 & 0.5781250000 \end{bmatrix}$$

$$x := \begin{bmatrix} 100.6328125 & 0.6835937500 \end{bmatrix}$$

$$x := \begin{bmatrix} 100.4746094 & 0.7626953125 \end{bmatrix}$$

(3)

Teil (b):

```
> print(bj);
ewj := eigenvalues(bj);
rhoj:=evalf(max(op(map(abs,{ewj})))));
```

$$\begin{bmatrix} 0 & -\frac{3}{2} \\ -\frac{1}{2} & 0 \end{bmatrix}$$

$$ewj := \frac{1}{2} \sqrt{3}, -\frac{1}{2} \sqrt{3}$$

$$rhoj = 0.8660254040 \quad (4)$$

```
> print(bgs);
ewgs := eigenvalues(bgs);
rhogs=evalf(max(op(map(abs,{ewgs}))));
```

$$\begin{bmatrix} 0 & -\frac{3}{2} \\ 0 & \frac{3}{4} \end{bmatrix}$$

$$ewgs := 0, \frac{3}{4}$$

$$rhogs = 0.7500000000 \quad (5)$$

Teil (c):

```
> omega_optimal := 2/(2 - min(ewj) - max(ewj));
```

$$omega_optimal := 1 \quad (6)$$

omega_optimal ist 1, also bringt die Relaxation nichts

Aufgabe 3

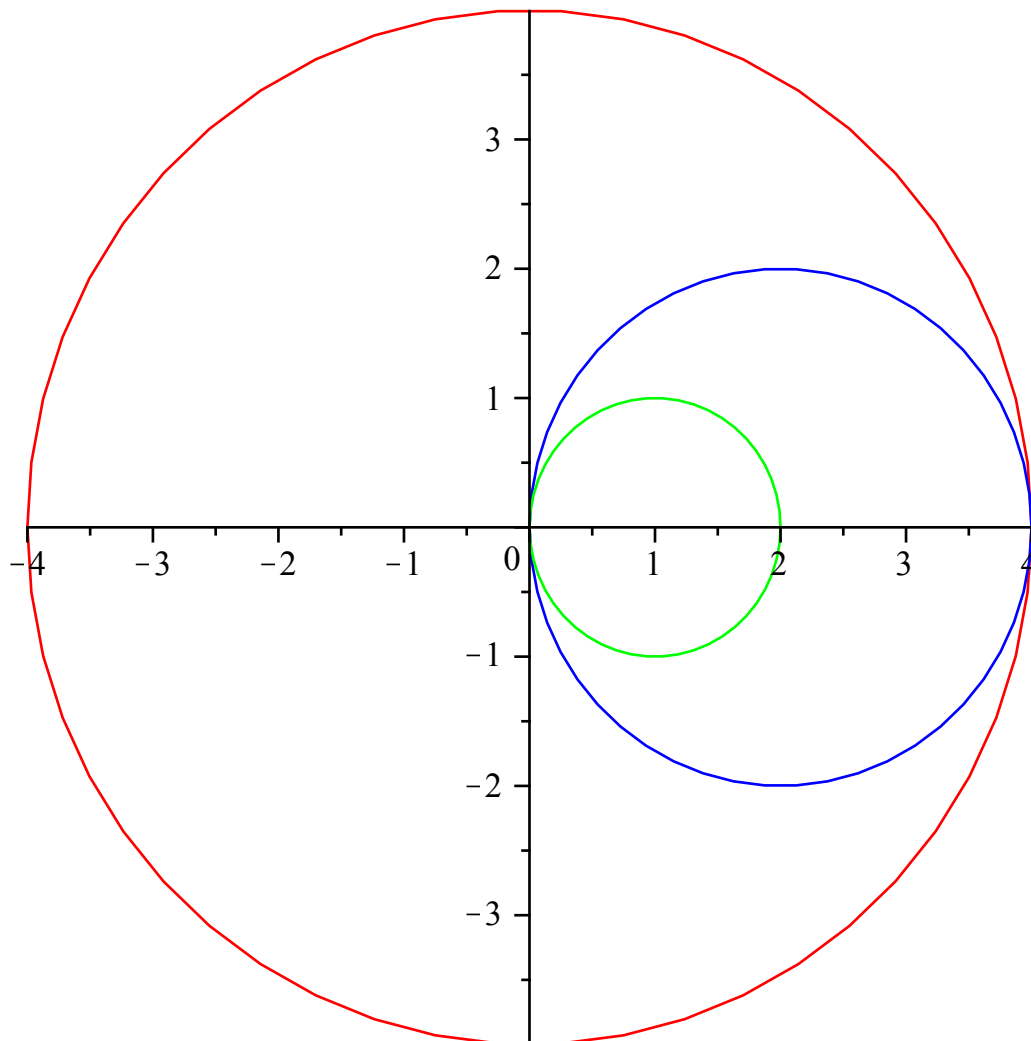
```
> restart:
with(LinearAlgebra):
with(plottools):with(plots):A:=Matrix([[0, -4, 0], [0, 2, -2],
[0, -1, 1]]);
```

$$A := \begin{bmatrix} 0 & -4 & 0 \\ 0 & 2 & -2 \\ 0 & -1 & 1 \end{bmatrix}$$

(1)

> Teil a)

```
> k1 := circle([0,0],4, color=red):k2 := circle([2,0],2, color=
blue):k3 := circle([1,0],1, color=green):display(k1,k2,k3);
```



> Teil b)

```
> Eigenvalues(A);
```

$$\begin{bmatrix} 0 \\ 0 \\ 3 \end{bmatrix}$$

(2)

> Teil c)

```
> vektorit:=proc(A,x)
local y,mue;
y:=A.x;
mue:=DotProduct(x,y);print(mue);
y/Norm(y,2);
```

end:

```
> n:=4;  
x:=evalf(<0,1,0>);  
for i from 1 to n do  
  x:=vektorit(A,x);  
od;
```

$n := 4$

$$x := \begin{bmatrix} 0. \\ 1. \\ 0. \\ 2. \end{bmatrix}$$

$$x := \begin{bmatrix} -0.8728715608000000057 \\ 0.4364357804000000028 \\ -0.2182178902000000014 \\ 2.23809523735694560 \end{bmatrix}$$

$$x := \begin{bmatrix} -0.766261028287208323 \\ 0.574695771215406159 \\ -0.287347885607703080 \\ 3.000000000086357366 \end{bmatrix}$$

$$x := \begin{bmatrix} -0.766261028210582285 \\ 0.574695771157936686 \\ -0.287347885578968343 \\ 3.00000000026357450 \end{bmatrix}$$

$$x := \begin{bmatrix} -0.766261028133956357 \\ 0.574695771100467212 \\ -0.287347885550233606 \end{bmatrix}$$

(3)

```
function x = bisektion(f,a,b,n)

for k = 1:n
    x = (a + b) / 2;

    fx = feval(f,x);
    fa = feval(f,a);

    if (fx*fa) < 0
        b = x;
    else
        a = x;
    end
end

end
```

Aufgabe 5

```
> restart:
f:=x->sin(x)-cos(x);
```

$$f := x \rightarrow \sin(x) - \cos(x)$$

(1)

Existenz einer Nullstelle: vorzeichen von $f(a)$ und $f(b)$ müssen unterschiedlich sein

```
> a:=0:
b:=2:
print(evalf(f(a)),evalf(f(b)));
```

$$-1., 1.32544426337$$

(2)

Bisektion

```
> for i from 1 to 3 do
  x:=(a+b)/2:
  print(evalf(x),evalf(f(x)));
  if(evalf(f(x)*f(a)))<0 then
    b:=x
  else
    a:=x
  fi:
  print(evalf(a),evalf(b));
od:
```

$$1., 0.301168678940$$
$$0., 1.$$
$$0.500000000000, -0.398157023286$$
$$0.500000000000, 1.$$
$$0.750000000000, -0.050050108851$$
$$0.750000000000, 1.$$

(3)

Newton

Startwert in Intervallmitte

```
> x:=evalf((a+b)/2);
```

$$x := 0.875000000000$$

(4)

Iteration

```
> Digits:=12:
phi:=x->x-f(x)/D(f)(x):
for i from 1 to 3 do
  x:=phi(x);
od;
```

$$x := 0.785157601706$$
$$x := 0.785398163402$$
$$x := 0.785398163397$$

(5)

Aufgabe 6

```
> restart:
with(LinearAlgebra):
x:=Vector([-1, 0, 1, 3]);
y:=Vector([5, 3, 5, 21]);
```

$$x := \begin{bmatrix} -1 \\ 0 \\ 1 \\ 3 \end{bmatrix}$$

$$y := \begin{bmatrix} 5 \\ 3 \\ 5 \\ 21 \end{bmatrix}$$

(1)

Teil (a): Newton-Darstellung, dividierte Differenzen

```
> with(LinearAlgebra):
n:=nops(x);
dd:=Matrix(n+1):
for i from 1 to n+1 do
  dd[i,1]:=y[i]:
  for j from 2 to i do
    dd[i,j]:=(dd[i,j-1]-dd[i-1,j-1])/(x[i]-x[i-j+1])
  od:
od:
print(dd);
```

$$n := 3$$

$$\begin{bmatrix} 5 & 0 & 0 & 0 \\ 3 & -2 & 0 & 0 \\ 5 & 2 & 2 & 0 \\ 21 & 8 & 2 & 0 \end{bmatrix}$$

(2)

Einzelteile aufsummieren

```
> ipn:=0:h:=1:
for i from 1 to n+1 do
  ipn:=ipn+dd[i,i]*h:
  h:=h*(t-x[i])
od:
ipn;
ipn:=unapply(expand(ipn),t);
```

$$3 - 2t + 2(t+1)t$$

$$ipn := t \rightarrow 3 + 2t^2$$

(3)

Teil (b): Ausgleichsproblem, enthält nur konstanten und quadratischen Anteil

```
> n:=Dimension(x):
phi:=[t->1,t->t^2]:
m:=nops(phi):
A:=map(phi[1],x):
for j from 2 to m do
  A:=<A|map(phi[j],x)>:
od:
```

```

A;
b:=y;

ATA:=MatrixMatrixMultiply(Transpose(A),A);
ATb:=MatrixVectorMultiply(Transpose(A),b);
ab:=LinearSolve(ATA,ATb);

p:=unapply(DotProduct(ab,Vector(apply(phi,t))),t);

```

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 9 \end{bmatrix}$$

(4)

$$b := \begin{bmatrix} 5 \\ 3 \\ 5 \\ 21 \end{bmatrix}$$

$$ATA := \begin{bmatrix} 4 & 11 \\ 11 & 83 \end{bmatrix}$$

$$ATb := \begin{bmatrix} 34 \\ 199 \end{bmatrix}$$

$$ab := \begin{bmatrix} 3 \\ 2 \end{bmatrix}$$

$$p := t \rightarrow 3 + 2t^2$$

```

> fehler := Norm(A.ab-b,2);

```

fehler := 0

(5)

Aufgabe 7

```
> restart:
with(student):
> f:=x->x/(x+1);
```

$$f:=x \rightarrow \frac{x}{x+1}$$

(1)

exakter Wert

```
> a:=0:
b:=4:
exakt:=evalf(int(f(s),s=a..b));
exakt:= 2.390562088
```

(2)

Anzahl der Trapeze für die verschiedenen Trapezregeln

```
> n := 2:
nt:=array(0..n):
for i from 0 to n do
    nt[i]:=2^i
od;
```

$$nt_0 := 1$$

$$nt_1 := 2$$

$$nt_2 := 4$$

(3)

zugehörige Trapezbreiten

```
> h:=map(nn->(b-a)/nn,nt);
h:= ARRAY ([0..2], [(0) = 4, (1) = 2, (2) = 1])
```

(4)

Trapezregeln ausrechnen

```
> t:=map(nn->value(trapezoid(f(x), x=a..b, nn)),nt);
t:= ARRAY ([0..2], [(0) =  $\frac{8}{5}$ , (1) =  $\frac{32}{15}$ , (2) =  $\frac{139}{60}$ ])
```

(5)

```
>
```

Aufgabe 8

```
> restart;
  Digits:=16;
```

Integrand

```
> f:=t->1/(1+t^2);
```

$$f := t \rightarrow \frac{1}{1+t^2} \quad (1)$$

Teil (a): Stammfunktion und exakter Wert

```
> F:=unapply(int(f(t),t),t);
  exakt:=F(1)-F(0);
```

$$F := t \rightarrow \arctan(t)$$

$$exakt := \frac{1}{4} \pi \quad (2)$$

Teil (b):

```
> o:=n->sum(f(i/n),i=0..(n-1))/n;
  u:=n->sum(f(i/n),i=1..n)/n;
  d:=n->(f(0)-f(1))/n;
```

$$o := n \rightarrow \frac{\sum_{i=0}^{n-1} f\left(\frac{i}{n}\right)}{n}$$

$$u := n \rightarrow \frac{\sum_{i=1}^n f\left(\frac{i}{n}\right)}{n}$$

$$d := n \rightarrow \frac{f(0) - f(1)}{n} \quad (3)$$

Teil (c):

```
> evalf(o(10));evalf(u(10));evalf(d(10));
  0.8099814972267897
  0.7599814972267897
  0.050000000000000000
```

(4)

Teil (d):

```
> evalf(exakt);
  0.7853981633974482
```

(5)

damit die ersten 3 Nachkommastellen stimmen muss der Fehler d(n) <= Pi/4-0.785 sein:

```
> solve(d(n)=exakt-0.785,n);
  1255.765856942968
```

(6)

Teil (e):

```
> simpson:=(f,a,b)->(b-a)/6*(f(a)+4*f((a+b)/2)+f(b));
  einfach:=evalf(simpson(f,0,1));
  fehler:=evalf(abs(exakt-einfach));
```

$$simpson := (f, a, b) \rightarrow \frac{1}{6} (b-a) \left(f(a) + 4f\left(\frac{1}{2}a + \frac{1}{2}b\right) + f(b) \right)$$

$$einfach := 0.7833333333333333$$

$$fehler := 0.0020648300641149$$

(7)

Teil (f):

```
> simpson:=proc(f,a,b,ms)
  local ls,h,i;
  if (type(ms,odd)) then print("Fehler") fi;
  ls:=(b-a)/ms;
  h:=f(a);
  for i from 1 to ms-1 do
    if (type(i,odd)) then h:=h+4*f(a+i*ls) else h:=h+2*f(a+i*ls)
  fi
  od:
  h:=h+f(b);
  ls/3*h;
end proc;
summiert:=evalf(simpson(f,0,1,10));
fehler:=evalf(abs(exakt-summiert));
```

summiert := 0.7853981534848038

fehler := 9.9126444 10⁻⁹

(8)

Teil (g):

```
> g1:=(f,a,b)->(b-a)/2*
  (f((a+b)/2-(b-a)/2/sqrt(3))+f((a+b)/2+(b-a)/2/sqrt(3)));
gauss:=evalf(g1(f,0,1));
fehler:=evalf(abs(exakt-gauss));
```

$$g1 := (f, a, b) \rightarrow \frac{1}{2} (b-a) \left(f \left(\frac{1}{2} a + \frac{1}{2} b - \frac{1}{2} \frac{b-a}{\sqrt{3}} \right) + f \left(\frac{1}{2} a + \frac{1}{2} b + \frac{1}{2} \frac{b-a}{\sqrt{3}} \right) \right)$$

gauss := 0.7868852459016396

fehler := 0.0014870825041914

(9)

Aufgabe 9

```
> restart;
gauss:=(f,a,b)->sum(w[i]*f(x[i]),i=1..n);
```

$$gauss := (f, a, b) \rightarrow \sum_{i=1}^n w_i f(x_i) \quad (1)$$

```
> n:=2;a:=-1;b:=1;
```

$$\begin{aligned} n &:= 2 \\ a &:= -1 \\ b &:= 1 \end{aligned} \quad (2)$$

Teil (a): Gleichungen aufstellen und durch einsetzen zeigen, dass die angegebenen Werte eine Lösung sind

```
> gleichungen:=NULL;
for j from 0 to 2*n-1 do
  f:=t->t^j;
  gleichungen:=gleichungen,int(x^j,x=a..b)=gauss(f,a,b):
od:
print(gleichungen);
lsg:=evalf(solve({gleichungen}));
```

$$\begin{aligned} 2 = w_1 + w_2, 0 = w_1 x_1 + w_2 x_2, \frac{2}{3} = w_1 x_1^2 + w_2 x_2^2, 0 = w_1 x_1^3 + w_2 x_2^3 \\ lsg := \{w_1 = 1., w_2 = 1., x_1 = 0.5773502693, x_2 = -0.5773502693\} \end{aligned} \quad (3)$$

Teil (b):

```
> f:=x->exp(x);
exakt:=int(f(x),x=a..b);evalf(exakt);
```

$$\begin{aligned} f &:= x \rightarrow e^x \\ exakt &:= -e^{-1} + e \\ &2.350402387 \end{aligned} \quad (4)$$

Ergebnisse aus (a) anwenden

```
> assign(lsg);
g:=gauss(f,a,b):evalf(g);
gfehler:=evalf(abs(exakt-g));
2.342696088
gfehler:=0.007706299
```

(5)

Teil (c):

```
> trapez:=(f,a,b)->(b-a)/2*(f(a)+f(b));
teinfach:=trapez(f,a,b):evalf(teinfach);
tfehler:=evalf(abs(exakt-teinfach));
```

$$trapez := (f, a, b) \rightarrow \frac{1}{2} (b - a) (f(a) + f(b))$$

$$3.086161269$$

$$tfehler := 0.7357588824$$

(6)

```
> simpson:=(f,a,b)->(b-a)/6*(f(a)+4*f((a+b)/2)+f(b));
seinfach:=simpson(f,a,b):evalf(seinfach);
sfehler:=evalf(abs(exakt-seinfach));
```

$$simpson := (f, a, b) \rightarrow \frac{1}{6} (b - a) \left(f(a) + 4f\left(\frac{1}{2}a + \frac{1}{2}b\right) + f(b) \right)$$

$$2.362053756$$

$$sfehler := 0.011651369$$

(7)

Teil (d):

obere Schranke für den Fehler muss kleiner als der Fehler der Gauss-Integration sein

```
> M:=maximize(abs(D@@2)(f)(x)),x=a..b);
-1*isolate(M*(b-a)^3/(12*m^2)<=gfehler,m);
grenze:=evalf(lhs(%));
```

$$\frac{M:=e}{\sqrt{\frac{1}{e}}} \leq |m|$$

$grenze := 15.33482005$

(8)

aufrunden

> **m:=ceil(%);**

$m := 16$

(9)

Teil (e): summierte Trapez-Regel

```
> trapez:=proc(f,a,b,m)
  local l,h,i:
  l:=(b-a)/m:
  h:=f(a);
  print(evalf(f(a)));
  for i from 1 to m-1 do h:=h+2*f(a+i*l);print(evalf(f(a+i*l)))
od:
  print(evalf(f(b)));
  h:=h+f(b):
  l/2*h:
end proc:
t:=trapez(f,a,b,m):
```

0.3678794412
 0.4168620197
 0.4723665527
 0.5352614285
 0.6065306597
 0.6872892788
 0.7788007831
 0.8824969026
 1.
 1.133148453
 1.284025417
 1.454991415
 1.648721271
 1.868245957
 2.117000017
 2.398875294
 2.718281828

(10)

```
> evalf(t);
tfehler:=evalf(abs(exakt-t));
```

2.353462011

$tfehler := 0.0030596234$

(11)

Teil (f): Streifenzahl aus Teil (d)/(e)

```
> simpson:=proc(f,a,b,ms)
  local ls,h,i:
  if (type(ms,odd)) then print("Fehler") fi;
  ls:=(b-a)/ms:
  h:=f(a);
  for i from 1 to ms-1 do
    if (type(i,odd)) then h:=h+4*f(a+i*ls) else h:=h+2*f(a+i*ls)
  fi
  od:
  h:=h+f(b):
  ls/3*h:
end proc:
s:=evalf(simpson(f,a,b,m));
fehler:=evalf(abs(exakt-s));
```

|

$$\begin{aligned}s &:= 2.350405569 \\fehler &:= 0.000003182\end{aligned}$$

(12)

Aufgabe 10

Differentialgleichung, Anfangswert und exakte Lösung

```
> restart;
f:=(t,x)->x^2;
dgl:=diff(x(t),t)=f(t,x(t));
aw:=x(0)=0.1;
x:=unapply(rhs(dsolve({dgl,aw},x(t))),t);
```

$$f := (t, x) \rightarrow x^2$$

$$dgl := \frac{d}{dt} x(t) = x(t)^2$$

$$aw := x(0) = 0.1$$

$$x := t \rightarrow -\frac{1}{t-10}$$

(1)

Teil (a):

```
> eu:=proc(f,xf,tf,h,n)
  local i,t,x,tt,xx:

  t:=tf;x:=xf;
  tt:=t;xx:=x;

  for i from 1 to n do
    x:=x+h*f(t,x):
    t:=t+h;
    tt:=tt,t:
    xx:=xx,x;
  od:
  [[tt],[xx]]:
end proc:
> tx:=eu(f,x(0),0,1,4):
tk:=tx[1];
xeuler:=evalf(tx[2]);
xexakt:=evalf(map(x,tx[1]));
```

$$tk := [0, 1, 2, 3, 4]$$

$$xeuler := [0.1000000000, 0.1100000000, 0.1221000000, 0.1370084100, 0.1557797144]$$

$$xexakt := [0.1000000000, 0.1111111111, 0.1250000000, 0.1428571429, 0.1666666667]$$

(2)

Teil (b):

```
> rk:=proc(f,xf,tf,h,n)
  local i,t,x,tt,xx,k1,k2,k3,k4:

  t:=tf;x:=xf;
  tt:=t;xx:=x;

  for i from 1 to n do
    k1:=f(t,x):
    k2:=f(t+h/2,x+h/2*k1):
    k3:=f(t+h/2,x+h/2*k2):
    k4:=f(t+h,x+h*k3):

    print(evalf([k1,k2,k3,k4]));
    x:=x+h/6*(k1+2*k2+2*k3+k4):
    t:=t+h;
    tt:=tt,t;xx:=xx,x;
  od:
  [[tt],[xx]]:
end proc:
```

```
> tx:=rk(f,x(0),0,4,1):  
tk:=tx[1];  
xrunge:=evalf(tx[2]);  
xexakt:=evalf(map(x,tx[1]));
```

```
[0.01000000000, 0.01440000000, 0.01658944000, 0.02767490431]
```

```
tk:= [0, 4]
```

```
xrunge:= [0.1000000000, 0.1664358562]
```

```
xexakt:= [0.1000000000, 0.1666666667]
```

(3)